🎯 Kubernetes Application Deployment Concepts – Interview Notes

1️⃣ Secrets 🔐

- **Purpose:** Store sensitive information (DB passwords, API keys) in an **encrypted format** rather than plain text in environment variables.
- **Use Case:** MySQL root password or application DB credentials.
- **Kubernetes Objects:** `Secret`
- **Types of Secrets:**
  - `Opaque` – generic key-value pairs
  - `docker-registry` – for pulling images from private registries
  - `tls` – for TLS certificates
- **Usage in Deployment:**

```
env:
  - name: MYSQL_ROOT_PASSWORD
    valueFrom:
      secretKeyRef:
        name: mysql-secret
        key: password
```

- ✅ **Why it's important:** Prevents sensitive data exposure in manifests or logs.

2️⃣ Init Containers 🛠️

- **Purpose:** Run **pre-start tasks** before the main application container starts.
- **Problem solved:** Ensures dependent services (like MySQL) are ready before the app starts.
- **Scenario:** App crashes/restarts because DB isn't ready.
- **ASCII Diagram:**

```
[Init Container] ---> [Wait for MySQL Ready] ---> [App Container Starts]
```

- **Key Points:**
  - Runs to completion before main containers start
  - Useful for **migrations, DB setup, config prep**

3️⃣ Liveness & Readiness Probes ❤️

- **Purpose:** Monitor container health and manage traffic routing.

Liveness Probe

- Checks if container is **alive**
- If it fails, Kubernetes **restarts the container**
- Example:

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

Readiness Probe

- Checks if container is **ready to serve traffic**
- If it fails, traffic **will not be sent** to the pod
- Example:

```
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 3
```

- **ASCII Diagram:**

```
Pod Lifecycle:
[Starting] -> [Ready?] -> [Serving Traffic] -> [Unhealthy?] -> [Restart]
```

4️⃣ Resource Requests & Limits 🖥️⚡

- **Purpose:** Control **CPU & memory usage** of containers in a cluster.
- **Request:** Minimum guaranteed resources
- **Limit:** Maximum allowed resources
- **Example:**

```
resources:
  requests:
```

```
      memory: "512Mi"
      cpu: "500m"
    limits:
      memory: "1Gi"
      cpu: "1"
```

- **Why important:** Prevents resource starvation and ensures fair allocation among applications.

5️⃣ Namespaces 📦

- **Purpose:** Logical separation of environments and resources within a single cluster.
- **Use Cases:**
  - Environment separation: `dev`, `qa`, `staging`, `prod`
  - Resource allocation per namespace (CPU, memory quotas)
  - Multi-team cluster usage without interference
- **Example:**

```
kubectl create namespace dev
kubectl create namespace qa
kubectl create namespace staging
```

- **ASCII Diagram:**

```
Cluster
├── Namespace: dev      -> 5 pods, 2 CPU
├── Namespace: qa       -> 3 pods, 1 CPU
└── Namespace: staging  -> 2 pods, 1 CPU
```

✅ Key Takeaways for Interviews

- Secrets = secure sensitive info
- Init containers = pre-start tasks
- Liveness probe = keep container alive
- Readiness probe = control traffic flow
- Requests & limits = CPU/memory control
- Namespaces = environment & resource separation

Pro Tip: Be ready to **draw diagrams** on a whiteboard showing init container flow, probe checks, and namespace segregation. Visual explanation often impresses interviewers.

🎯 Kubernetes Secrets – Interview Notes

1️⃣ What are Kubernetes Secrets? 🔐

- **Definition:** Kubernetes Secrets let you **store and manage sensitive data** such as passwords, tokens, or SSH keys in **Base64 encoded format**.
- **Why use Secrets?**
  - Safer than hardcoding credentials in manifests
  - Flexible and reusable across multiple deployments
  - Keeps sensitive info out of Git or logs

2️⃣ Base64 Encoding 🏷️

- Secrets require **Base64 encoding** of values.
- **Ways to encode:**
  - Linux/macOS: `echo -n "DBpassword11" | base64`
  - Windows: Use online tool → [base64encode.org]
- **Example:**

```
Original: DBpassword11
Base64: RERwYXNzd29yZDEx
```

3️⃣ Creating a Secret Manifest 📄

- **Standard Template:**

```
apiVersion: v1
kind: Secret
metadata:
  name: mysql-db-password    # Name of the secret
type: Opaque                 # Unstructured key-value data
data:
  db-password: RERwYXNzd29yZDEx  # Base64 encoded password
```

- **Key Points:**

- `type: Opaque` → Unstructured key-value pairs
- `data` → contains encoded sensitive info
- You can add multiple keys: username, password, host, etc.

4️⃣ Using Secrets in Deployments 🚀

- Instead of hardcoding environment variables, use **`valueFrom: secretKeyRef`**
- **Example – MySQL Deployment:**

```
env:
  - name: MYSQL_ROOT_PASSWORD
    valueFrom:
      secretKeyRef:
        name: mysql-db-password   # Secret name
        key: db-password          # Key inside secret
```

- Same method applies for **microservices** that need DB credentials:

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: mysql-db-password
        key: db-password
```

- ✅ Keeps sensitive data centralized and secure

5️⃣ Applying Secrets and Deployments ⚡

- **Steps:**
  1. Create the Secret manifest: `kubectl apply -f secret.yaml`
  2. Update deployments to reference the Secret
  3. Apply updated deployments: `kubectl apply -f deployment.yaml`
  4. Verify pods:

     ```
     kubectl get pods
     kubectl logs <pod-name>
     ```

- **Note:** If the DB password is wrong, pods fail to start → always check `kubectl logs`.

6️⃣ Observations & Best Practices ✅

- After applying secrets:
  - MySQL container and dependent services start correctly
  - User management service can successfully connect to DB
- **Key Takeaways:**
  - Secrets prevent exposing passwords in plain text
  - Works for any sensitive config (API keys, SSH keys, etc.)
  - Use Base64 encoding for Kubernetes compatibility
- **Tip:** Always test by restarting pods to ensure Secrets work correctly

---

7️⃣ ASCII Diagram – Secret Flow 🔄

```
[Secret: mysql-db-password]
        |
        v
+-----------------+
| Deployment Env  |
| MYSQL_ROOT_PASS |
+-----------------+
        |
        v
  [Pod starts with password securely injected]
```

8️⃣ Common Interview Questions 💡

- **Q:** Why use Secrets over ConfigMaps?
  A: Secrets store sensitive info securely; ConfigMaps are plain text.
- **Q:** What is `Opaque` type in Secrets?
  A: It means the data is unstructured key-value pairs, not tied to a specific format.
- **Q:** How to reference a Secret in a Deployment?
  A: Use `valueFrom: secretKeyRef` with the secret name and key.

✅ **Pro Tip for Interviews:** Be ready to **draw the Secret flow diagram** and explain Base64 encoding & usage in deployments. Interviewers often test practical understanding, not just theory.

Exactly! ✅ You can manage Kubernetes secrets in multiple ways, not just the basic `Secret` objects. Let me break it down clearly for interviews:

1️⃣ Native Kubernetes Secrets 🔐

- Stored **inside the cluster** as Base64-encoded data
- Easy to create and reference (`kubectl create secret`, `valueFrom: secretKeyRef`)
- **Limitations:**
  - Stored in etcd, so extra encryption is needed for production
  - Manual rotation can be tricky

2️⃣ HashiCorp Vault Integration 🏰

- **HashiCorp Vault** is a **secret management tool** that provides dynamic secrets and centralized control
- Features:
  - Secrets are **dynamically generated** (e.g., DB creds created on-the-fly)
  - Secrets can **auto-rotate** or have TTL (time-to-live)
  - Strong **access control** and auditing
- Kubernetes Integration:
  - Use **Vault Agent Injector** or **Vault CSI Driver** to inject secrets as files or env vars into pods
- **Diagram:**

```
[Vault Server] ---> [Kubernetes CSI Driver] ---> [Pod Env/File]
```

3️⃣ AWS Secrets Manager Integration ☁️

- Fully managed secret storage by AWS
- Features:

    - Secrets are **encrypted by AWS KMS**
    - Can **rotate automatically**
    - Securely inject secrets into **EKS pods** using `Secrets Store CSI Driver` or SDK
- Kubernetes Workflow:
    - Store secret in AWS Secrets Manager
    - Use **Secrets Store CSI Driver** in EKS
    - Pod mounts the secret as a **volume or env var**
- **ASCII Diagram:**

```
[AWS Secrets Manager] --CSI Driver--> [Kubernetes Pod Env/Volume]
```

✅ Key Points for Interviews

- Kubernetes native secrets are **simpler** but have storage limitations
- **Vault** is better for dynamic, auditable secrets across environments
- **AWS Secrets Manager** is ideal if you're already in the AWS ecosystem
- Always emphasize **encryption, rotation, and least privilege** in answers

---

Absolutely! Let's break this down carefully. The **CSI driver** concept is key for production-grade Kubernetes secret management, and yes—AKS has a similar approach.

1️⃣ What is a CSI Driver? 📦

CSI = Container Storage Interface

Originally designed to **standardize how storage volumes are attached to Kubernetes pods**, it has been extended to also **inject secrets from external secret stores**.

- **Purpose in Secrets Management:**
    - Allows Kubernetes pods to **mount external secrets** (Vault, AWS Secrets Manager, Azure Key Vault) as **volumes or environment variables**.
    - Avoids storing sensitive information inside the cluster itself.
- **How it Works:**

```
[External Secret Store] --> [CSI Driver] --> [Pod Volume / Env]
```

- **Example:** Vault, AWS Secrets Manager, Azure Key Vault all have a **CSI driver implementation**.

---

2 AWS / EKS Example ☁

- Use **Secrets Store CSI Driver**:
  - Pod mounts a volume that **dynamically fetches secrets from AWS Secrets Manager**
  - Secrets can appear as **files in the container**
  - Automatic refresh supported

Diagram:

```
[AWS Secrets Manager]
        |
        v
[Secrets Store CSI Driver]
        |
        v
[Pod Volume / Env Variable]
```

- Advantages:
  - Secrets never stored in Kubernetes etcd
  - Can use IAM roles for access control
  - Automatic rotation possible

---

3 Azure / AKS Equivalent 🌐

- AKS uses **Azure Key Vault** + **Secrets Store CSI Driver for Azure**
- Workflow:
  1. Create secrets in **Azure Key Vault**
  2. Install **Azure Key Vault CSI Driver** in AKS
  3. Mount secrets as **volumes** in pods
- **Diagram:**

```
[Azure Key Vault]
        |
        v
[Azure Key Vault CSI Driver]
        |
```

```
        v
[Pod Volume / Env Variable]
```

- Benefits:
    - Same as AWS: external storage, secure, dynamic
    - Works seamlessly with AKS RBAC & managed identities

4️⃣ Why CSI is Recommended for Interviews 🎯

- Shows **cloud-native production knowledge**
- Explains **dynamic, secure secret injection**
- Demonstrates understanding beyond basic Kubernetes Secrets

Quick Talking Points:

- "CSI driver allows mounting secrets from external stores as **volumes**"
- "Reduces risk of storing sensitive data in etcd"
- "Supports automatic secret rotation and cloud-native IAM integration"

Exactly! ✅ In **EKS (Amazon Elastic Kubernetes Service)**, using the **Secrets Store CSI Driver** is the modern, production-ready way to inject secrets from **AWS Secrets Manager** or **AWS Systems Manager Parameter Store** into your pods.

Here's how it works in practice:

1️⃣ Workflow in EKS 🌐

1. **Create secrets in AWS**
    - Example: DB password in **Secrets Manager**
    - Optionally enable **automatic rotation**
2. **Install the Secrets Store CSI Driver in your EKS cluster**

    ```
    kubectl apply -f https://github.com/kubernetes-sigs/secrets-store-csi-driver/deploy/rbac-secretproviderclass.yaml
    ```

    - This allows pods to mount external secrets as volumes
3. **Create a SecretProviderClass**
    - Defines **which secret to fetch** and **from where**

    ```
    apiVersion: secrets-store.csi.x-k8s.io/v1
    kind: SecretProviderClass
    ```

```
metadata:
  name: aws-secrets
spec:
  provider: aws
  parameters:
    objects: |
      - objectName: "my-database-password"
        objectType: "secretsmanager"
```

4. **Mount secrets in pods**

```
volumes:
  - name: secret-vol
    csi:
      driver: secrets-store.csi.k8s.io
      readOnly: true
      volumeAttributes:
        secretProviderClass: "aws-secrets"
```

5. **Use the secrets in your application**
   - The secret appears as a **file inside the pod**
   - Can also be projected as **environment variables**

2️⃣ Benefits of this approach for EKS ✅

- Secrets are **not stored in Kubernetes etcd** → safer
- Supports **dynamic secret rotation**
- Works seamlessly with **IAM roles for service accounts (IRSA)**
- Centralizes secret management in **AWS Secrets Manager**

🔷 ASCII Diagram – EKS Secret Injection

```
[AWS Secrets Manager]
        |
        v
[Secrets Store CSI Driver] ---> [Mounted Volume in Pod]
        |
```

```
        v
[App reads DB password securely]
```

So yes, in **EKS**, you **don't have to manually create Kubernetes Secrets** for production credentials. Instead, you rely on the **CSI driver** to dynamically fetch secrets from AWS Secrets Manager (or SSM Parameter Store) and mount them securely in pods.

Perfect! I've turned your transcript into **interview-ready notes on Kubernetes Init Containers**, expanded with explanations, diagrams, and practical tips.

🎯 Kubernetes Init Containers – Interview Notes

1️⃣ What are Init Containers? 🛠️

- **Definition:** Special containers that **run before the main app container** in a Pod.
- **Purpose:**
  - Perform setup tasks that the main container cannot do
  - Contain utilities or scripts not included in the main app image
  - Ensure dependencies (e.g., databases) are ready before the app starts
- **Key Characteristics:**
  - Run **to completion** before app containers start
  - Multiple init containers can be defined **sequentially**
  - Pod won't start if any init container **fails**
  - If restartPolicy is `Always` or `OnFailure`, Kubernetes **retries the pod** until init containers succeed

---

2️⃣ How Init Containers Work 🔄

- **Sequential Execution:**

```
[Init Container 1] --> [Init Container 2] --> ... --> [App Container]
```

- Each container must **complete successfully** before the next starts
- If an init container fails: Pod restarts (unless restartPolicy=Never)

3️⃣ Use Case Example – MySQL Dependency ✅

- **Scenario:** User management microservice depends on MySQL
- **Problem:** App starts **before MySQL is ready**, causing failures
- **Solution:** Init container checks MySQL availability before starting app
- **Implementation:**

```
initContainers:
  - name: init-db
    image: busybox
    command:
      - /bin/sh
      - -c
      - |
        until nc -z mysql 3306; do
          echo "Waiting for MySQL..."
          sleep 2
        done
        echo "MySQL DB server is up"
```

- **Explanation:**
  - Uses **NetCat (nc)** to check if MySQL service is reachable on port 3306
  - Loops until MySQL is ready
  - Prints status and **terminates once complete**

4️⃣ Pod Spec Structure 🏗️

- **Pod Template Example:**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-management
spec:
  replicas: 1
  template:
    spec:
```

```
  initContainers:          # Init containers run first
    - name: init-db
      image: busybox
      command: ["/bin/sh", "-c", "until nc -z mysql 3306; do sleep 2; done"]
  containers:              # Main app container
    - name: user-management-app
      image: user-management:latest
```

- Level in YAML: **initContainers and containers are siblings under `spec`**

5️⃣ Observing Init Containers 🖥️

- **Check pod status:**

```
kubectl get pods
kubectl get pods -w   # watch mode
```

- **Check detailed pod info:**

```
kubectl describe pod <pod-name>
```

- **Events to look for:**
  - Init container starts
  - Checks for DB readiness
  - Completes → main app container starts
- **Key Point:** Once init container completes, **exit code = 0** and it **terminates**

6️⃣ Benefits of Init Containers ✅

- Ensures **dependency readiness** (DB, API, external services)
- Supports **multi-step pod initialization**
- Keeps **main app container image clean** (no extra utilities/scripts inside app image)
- Reduces **pod restarts** due to dependency issues

7️⃣ ASCII Diagram – Init Container Flow

```
Pod Start
   |
   v
```

```
[Init Container 1: Check MySQL]
   |
   v
[Init Container 2: Optional other setup]
   |
   v
[App Container: User Management Microservice]
```

8️⃣ Interview Talking Points

- "Init containers run **before the app container** and must complete successfully"
- "They help handle **dependency readiness** without bloating the main app image"
- "Useful in scenarios like waiting for DB, migrating schema, or pre-populating files"
- Can have **multiple sequential init containers**

💡 **Pro Tip:** In interviews, you can mention how **Secrets + Init Containers + Liveness/Readiness Probes** together ensure **secure and reliable application startup** in Kubernetes.

🎯 Kubernetes Probes – Interview Notes

Kubernetes probes are **health checks** for your containers that help Kubelet manage container lifecycle and traffic routing.

1️⃣ Types of Probes ⚡

Probe Type	Purpose	Behavior
Liveness	Detects if container is **alive or deadlocked**	If failed → container is restarted to get a fresh instance
Readiness	Detects if container is **ready to serve traffic**	If failed → container removed from service load balancer until it's ready
Startup	Detects if container has **finished starting**	While starting → disables liveness & readiness probes to avoid premature restarts

---

2️⃣ Liveness Probe

- Checks if the **container is alive**

- Common scenario: app deadlocks or gets stuck
- On failure → Kubernetes **restarts the container**

Example (HTTP-based liveness probe):

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

Key points:

- Ensures **self-healing** of pods
- Protects against frozen or deadlocked containers

---

3 Readiness Probe 🚦

- Checks if the **container is ready to accept traffic**
- Until it succeeds → pod is **not added to the Service load balancer**
- Prevents sending traffic to **uninitialized containers**

Example (TCP-based readiness probe):

```
readinessProbe:
  tcpSocket:
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 3
```

Key points:

- Useful for **slow-starting apps**
- Helps maintain **service availability**

---

4 Startup Probe 🏁

- Checks if **container has started successfully**
- Automatically disables **liveness and readiness probes** until startup probe succeeds
- Prevents **premature restarts** for **slow-starting applications**

Example (command-based startup probe):

```
startupProbe:
  exec:
    command: ["sh", "-c", "curl -f http://localhost:8080/health || exit 1"]
  failureThreshold: 30
  periodSeconds: 10
```

Key points:

- Ideal for apps that take **long to initialize**
- Avoids infinite restart loops caused by liveness probes

---

5 Ways to Define Probes 🛠️

- **Command (exec):** Run shell commands inside container

  ```
  nc -z localhost 8080  # checks if port is listening
  ```

- **HTTP GET:** Check a **health endpoint**

  ```
  httpGet:
    path: /health
    port: 8080
  ```

- **TCP Socket:** Check if a **port is open**

  ```
  tcpSocket:
    port: 8080
  ```

Tip: You can mix and match methods depending on your app's behavior.

---

6 ASCII Diagram – Probe Flow

```
         Pod Lifecycle
               |
      +-------+--------+
      |                |
  Startup Probe     App starts after success
      |
      v
  Readiness Probe
      |
```

```
     v
Accept traffic only if successful
     |
     v
Liveness Probe
     |
     v
Restart container if deadlocked
```

7️⃣ Interview Talking Points

- "**Liveness** → self-healing / restart if deadlocked"
- "**Readiness** → ensures traffic is only sent to ready containers"
- "**Startup** → avoids premature restarts for slow-starting apps"
- "Probes can be **HTTP GET, TCP socket, or command**"
- "Combining probes with **Secrets and Init Containers** ensures secure, reliable application startup in Kubernetes"

💡 **Pro Tip:**
In interviews, always mention how **readiness vs liveness** solves different problems:

- Readiness = **traffic control**
- Liveness = **container health/self-healing**
- Startup = **slow initialization handling**

Perfect! Here's a **condensed, interview-ready guide** based on your latest lecture about **defining Liveness and Readiness probes** in Kubernetes, with practical explanations and YAML examples.

Kubernetes Probes in Deployment Manifests

1️⃣ Key Concepts Recap

Probe Type	Purpose
Liveness	Detects if container is alive. On failure → container is **restarted**.
Readiness	Detects if container is ready to **accept traffic**. If not ready → pod is **removed from**

Probe Type	Purpose
	service load balancer.

Important fields in probes:

- **`initialDelaySeconds`** → Wait time after container starts before probe begins
- **`periodSeconds`** → Frequency of probe checks
- **`timeoutSeconds`** → How long to wait for a probe to respond (default 1s)
- **`successThreshold`** → Consecutive successes to mark healthy
- **`failureThreshold`** → Consecutive failures to mark unhealthy

---

2 Example: Liveness Probe using Command

```
livenessProbe:
  exec:
    command:
      - sh
      - -c
      - "nc -z localhost 8095"
  initialDelaySeconds: 60   # wait 60s for app to settle
  periodSeconds: 10         # check every 10s
  timeoutSeconds: 5
  failureThreshold: 3
  successThreshold: 1
```

Explanation:

- Uses **netcat (`nc -z`)** to check if port `8095` is listening
- Waits **60s** before first check to allow app startup
- Checks every **10s**, retries 3 times before marking as failed

---

3 Example: Readiness Probe using HTTP GET

```
readinessProbe:
  httpGet:
    path: /health
    port: 8095
  initialDelaySeconds: 60   # wait for app initialization
```

```
  periodSeconds: 10         # repeat check every 10s
  timeoutSeconds: 5
  failureThreshold: 3
  successThreshold: 1
```

Explanation:

- Hits the app's `/health` endpoint to verify readiness
- Ensures **traffic is only sent when app is truly ready**
- Initial delay ensures app isn't marked as "not ready" immediately after container start

---

4 Probe Behavior in Deployment

Timeline Example:

```
Time after container start:
0s       → Container starts
0-60s    → Waiting for initialDelaySeconds
60s      → Readiness probe begins
70s      → Liveness probe starts (checks every 10s)
60-70s   → Container shows "0/1 ready"
~95s     → Container becomes "1/1 ready" after passing readiness probe
```

- Liveness probe **won't restart** container if it's healthy
- Readiness probe **prevents traffic** until initial checks pass

---

5 Best Practices

- **Initial delay**: Always set for apps that need startup time
- **Period**: 5–10s is typical for most apps
- **Timeout**: Set slightly higher than app response time
- **Separate liveness vs readiness checks**:
  - Liveness = **app health**
  - Readiness = **app availability for traffic**

💡 **Interview Talking Points:**

- "Liveness checks for deadlocks; readiness checks for serving traffic."

- “Initial delay avoids marking the app as unhealthy before startup.”
- “Readiness probe ensures pod is registered with the service only when fully ready.”
- “Both probes can use **exec**, **HTTP GET**, or **TCP socket** depending on application.”

Here’s a clear, interview-friendly summary of **Kubernetes resource requests and limits**, based on your lecture:

💻 Kubernetes Container Resource Management: Requests & Limits

Kubernetes lets you define how much **CPU** and **memory** each container **requests** and the **maximum it can use**. This helps the scheduler and Kubelet manage cluster resources efficiently.

1️⃣ Key Concepts

Term	Description
Requests	Minimum resources a container **guarantees** to have. Scheduler uses this to decide **which node** to place the pod on.
Limits	Maximum resources a container is **allowed to use**. Kubelet enforces this at runtime.

Notes:

- 1000m CPU = 1 vCPU core
- Memory is usually in Mi (mebibytes), e.g., 128Mi = ~135MB
- Requests ensure your container **always has resources**, limits prevent it from **overusing** node capacity.

2️⃣ Example: Defining Requests & Limits in Deployment

```
resources:
  requests:
    memory: "128Mi"
    cpu: "500m"   # 0.5 vCPU
  limits:
    memory: "500Mi"
    cpu: "1"      # 1 vCPU
```

Explanation:

- `requests` → guarantees 128Mi memory and 0.5 vCPU
- `limits` → container cannot exceed 500Mi memory and 1 vCPU
- Scheduler uses **requests** to place pods
- Kubelet enforces **limits** to prevent resource overuse

---

3 Behavior on the Cluster

- If **node has insufficient resources** to satisfy a pod's requests → pod stays **Pending** until resources free up
- High traffic or scaling: auto-scaling can add nodes or adjust resources
- Important for **critical applications** to avoid being starved of CPU or memory

---

4 Viewing Resource Usage

```
kubectl describe node <node-name>
```

Example Output Interpretation:

- Shows requests and limits for all pods on the node
- Helps you understand **percentage of node resources already reserved**
- Plan your cluster capacity accordingly

---

5 Best Practices

1. **Always define requests** for critical apps → ensures guaranteed resources
2. **Set limits** to prevent a single container from hogging node resources
3. Monitor cluster and adjust **requests/limits** based on actual app behavior
4. Use **auto-scaling** for dynamic workloads

💡 **Interview Tip:**

- "Requests are for scheduling, limits are for runtime enforcement."
- "Define critical app requests conservatively; limits prevent resource abuse."
- "Without requests, Kubernetes can overcommit nodes, leading to OOM kills or CPU throttling."

Here's a structured summary of your lecture on **Kubernetes Namespaces**:

🌐 Kubernetes Namespaces: High-Level Overview

Namespaces in Kubernetes are **virtual clusters within a physical cluster**, providing **isolation** for resources, objects, and teams.

1️⃣ What Are Namespaces?

- Namespaces allow you to **organize and isolate Kubernetes objects** (Pods, Services, Deployments) within a single cluster.
- Think of them as **virtual clusters**: objects in one namespace cannot conflict with objects in another namespace.
- Useful for:
  - Multiple teams/projects sharing the same cluster.
  - Resource isolation and organization.
- Small/simple clusters can work without namespaces, but **using them is recommended** for better management.

2️⃣ Key Benefits

Benefit	Description
Isolation	Objects in one namespace are separate from others. Names can be reused across namespaces.
Resource Management	Use **ResourceQuota** to limit CPU/memory per namespace.
Team/Project Organization	Each project or environment can have its own namespace (e.g., `dev`, `qa`, `prod`).

3️⃣ Default Namespaces

Kubernetes creates a few namespaces automatically:

Namespace	Purpose
default	Default namespace for all objects if no namespace is specified.
kube-system	Contains **system-related components** (e.g., `kube-proxy`, `coreDNS`, CSI controllers).
kube-node-lease	Manages node heartbeat objects for performance.
kube-public	Readable by all users (even unauthenticated). Mainly for public cluster-wide info.

Example:

```
kubectl get namespaces
kubectl get all -n kube-system
kubectl get pods --namespace kube-system
```

---

4 Namespace Use Cases

- Deploy the **same app in different environments**:
  - `dev` namespace → dev backend + dev frontend
  - `qa` namespace → qa backend + qa frontend
- **Isolation example:** Two deployments named `backend` can coexist because they are in different namespaces.

---

5 Creating Namespaces

Declarative (YAML)

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev3
```

Imperative (kubectl)

```
kubectl create namespace dev1
```

Tip: Always specify a namespace for team/project deployments to maintain clarity and avoid conflicts.

---

6 How Namespaces Affect Objects

- Objects like Pods, Services, and Deployments **live inside a namespace**.
- Namespace scope determines:
  - Naming isolation (duplicate names allowed in different namespaces)
  - Resource quotas (CPU/memory)
  - Access control (RBAC policies)

Example:

- Deployment `backend` in `dev` namespace
- Deployment `backend` in `qa` namespace → both coexist without conflict

---

Namespaces are essential for **multi-team clusters, environment segregation, and resource management**.

Here's a structured summary of your lecture on **working with Kubernetes namespaces imperatively, deploying applications, and cleaning up**:

🌐 Kubernetes Namespaces: Practical Hands-On (Imperative Approach)

In this session, we worked with namespaces in **three ways**:

1. Imperative creation
2. Declarative YAML manifests
3. Resource management (LimitRange & ResourceQuota — to be covered later)

1️⃣ Creating Namespaces Imperatively

- Two namespaces created for separate environments:

  ```
  kubectl create namespace dev1
  kubectl create namespace dev2
  ```

- Verify namespaces:

  ```
  kubectl get namespaces
  # alias: kubectl get ns
  ```

Tip: Use namespaces to isolate environments like `dev1`, `dev2`, `qa`, etc.

2️⃣ Deploying Applications into Namespaces

- Standard Kubernetes objects (Pods, Deployments, Services) can be deployed into a specific namespace:

  ```
  kubectl apply -f cube-manifests/ -n dev1
  kubectl apply -f cube-manifests/ -n dev2
  ```

- Dynamic NodePort allocation:
  - NodePorts must be unique per cluster.
  - Comment static NodePort in YAML for multi-namespace deployment.
  - Kubernetes automatically assigns a free port dynamically.

3 Verifying Deployments

- Check all objects in a namespace:

  ```
  kubectl get all -n dev1
  kubectl get all -n dev2
  ```

- Persistent Volume Claims (PVCs) are **namespace-scoped**:

  ```
  kubectl get pvc -n dev1
  kubectl get pvc -n dev2
  ```

- Persistent Volumes (PVs) and Storage Classes (SC) are **cluster-scoped** and not tied to a namespace.

4 Accessing Applications

- Retrieve service NodePort and cluster IP:

  ```
  kubectl get svc -n dev1
  kubectl get svc -n dev2
  ```

- Use the public IP + NodePort to access applications:

  ```
  http://<public-ip>:<node-port>/user-mgmt/health
  ```

- Both dev1 and dev2 deployments run independently without conflicts.

5 Cleaning Up Namespaces & Objects

- **Delete objects in a namespace**:

  ```
  kubectl delete -f cube-manifests/ -n dev1
  ```

- **Delete an entire namespace** (automatically deletes all objects inside it):

  ```
  kubectl delete namespace dev1
  kubectl delete namespace dev2
  ```

- Persistent resources like Storage Classes remain **cluster-wide** and must be deleted separately:

  ```
  kubectl delete sc <storage-class-name>
  ```

- Verify cleanup:

  ```
  kubectl get all --all-namespaces
  kubectl get ns
  ```

✅ Key Takeaways

1. **Namespaces allow environment isolation** (e.g., dev1, dev2) without conflicts in object names.
2. **PVCs are namespace-scoped**, but PVs and StorageClasses are cluster-scoped.
3. **NodePorts must be unique**, so use dynamic assignment when deploying the same application across multiple namespaces.
4. **Deleting a namespace deletes all contained objects**, simplifying cleanup.
5. Using namespaces improves organization, multi-team collaboration, and environment separation.

This workflow demonstrates **the power of Kubernetes namespaces** in isolating environments and managing applications at scale.

Here's a clear breakdown of your lecture on **Kubernetes LimitRange in relation to namespaces**:

📌 Kubernetes LimitRange Concept

LimitRange is a Kubernetes feature that **enforces default resource limits and requests for containers within a namespace**, so you don't have to define them individually in each Deployment or Pod.

1️⃣ Why LimitRange is useful

- In Kubernetes, each container can have **resource requests** and **limits**:
  - **Requests** → Minimum CPU/memory the container needs to run.
  - **Limits** → Maximum CPU/memory the container can use.
- Without LimitRange, you must manually define these in every Deployment or Pod spec.
- LimitRange allows you to **automatically enforce defaults per namespace**, reducing manual work and ensuring consistency.

2️⃣ How LimitRange works

- Applied at the **namespace level**.
- Can define:
  1. **Default limits** – Maximum CPU/memory each container can use if not explicitly defined.

2. **Default requests** – CPU/memory a container initially requests if not defined.

- Example:
  1. Default Limit: 512MB memory, 0.5 CPU
  2. Default Request: 256MB memory, 0.3 CPU
- Any container in the namespace without explicit resource specs will automatically inherit these values.

---

3 LimitRange Manifest Structure

Example manifest for namespace `dev3`:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: dev3-limits
  namespace: dev3
spec:
  limits:
    - type: Container
      default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "300m"
        memory: "256Mi"
```

- `type: Container` → Applies to all containers in the namespace.
- `default` → Maximum resources a container can use.
- `defaultRequest` → Initial resources a container requests.

---

4 Practical Benefits

1. **Consistency** → All containers in a namespace get standard resource allocation.
2. **Reduced Manual Effort** → No need to update every Deployment with resource specs.
3. **Better Resource Management** → Prevents a container from consuming excessive resources, protecting the cluster from overload.

4. **Namespace-Level Enforcement** → Works across dev, QA, staging, and other environments.

5️⃣ How to Implement

1. Create namespace (if not already):

   `kubectl create namespace dev3`

2. Apply the LimitRange manifest:

   `kubectl apply -f dev3-limitrange.yaml`

3. Deploy Pods/Deployments in `dev3` without specifying resources. They will inherit defaults automatically.

💡 **Key Insight:**
LimitRange allows Kubernetes admins to enforce **resource discipline** at the namespace level, ensuring workloads run efficiently without needing to manually specify CPU/memory in every manifest.

Here's a clear summary and step-by-step explanation of your lecture on **creating namespaces and LimitRange manifests in Kubernetes**, with practical reasoning behind each step:

📌 LimitRange Default Concept – Step by Step

This lecture focuses on **declarative creation** of a namespace (`dev3`) and associating a LimitRange to enforce CPU/memory defaults for all containers in that namespace.

1️⃣ Why Put Namespace Manifest First

- All Kubernetes objects (Deployments, Services, PVCs, etc.) must exist **inside a namespace**.
- If you try to create other objects before the namespace exists, you get errors.
- Therefore, namespace manifest is placed as **`00-namespace.yaml`**.
- This allows you to deploy everything with **one command**:

```
kubectl apply -f .
```

No need to create the namespace separately first.

2️⃣ Namespace Manifest Template

```
apiVersion: v1
kind: Namespace
```

```
metadata:
  name: dev3
```

- **Kind:** `Namespace`
- **metadata.name:** `dev3`
- No spec is required.

---

3 LimitRange Manifest Template

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-cpu-mem-limit-range
  namespace: dev3
spec:
  limits:
    - type: Container
      default:
        cpu: "500m"
        memory: "512Mi"
      defaultRequest:
        cpu: "300m"
        memory: "256Mi"
```

- **metadata.namespace:** `dev3` → Ensures this LimitRange applies only to that namespace.
- **type: Container** → Applies limits to all containers in the namespace.
- **default** → Maximum CPU/memory per container.
- **defaultRequest** → Initial CPU/memory request per container.

---

4 Important Notes About Default Behavior

1. **Default limits if not specified:**
   - CPU → 1 vCPU
   - Memory → 512Mi
2. **Default requests if not specified:**

    - Kubernetes sets the request equal to the limit.
    - ⚠️ This can be dangerous because every container may assume it can consume maximum resources immediately.
3. **Best Practice:** Always specify both `default` and `defaultRequest` in LimitRange.

5️⃣ Updating Existing Manifests to Use Namespace

- Every object you deploy (Deployment, Service, PVC, ConfigMap, etc.) should include the **namespace** in its metadata:

```
metadata:
  name: user-management
  namespace: dev3
```

- **Exceptions:**
    - **StorageClass** → Cluster-level resource, not namespace-specific.

This ensures all resources go into the correct namespace (`dev3`) and adhere to the LimitRange automatically.

6️⃣ Steps Summary

1. **Create namespace manifest** → `00-namespace.yaml`
2. **Create LimitRange manifest** → `00-limitrange.yaml`
3. **Update all other manifests** with `namespace: dev3`
4. **Apply everything together**:

```
kubectl apply -f .
```

- Objects will automatically inherit the **LimitRange defaults** for CPU and memory.

✅ **Result:**

- Namespace `dev3` exists.
- LimitRange enforces resource limits and requests.
- All containers in `dev3` now have consistent, safe default resources.

Here's a structured summary of what this lecture covered, with key takeaways and commands for clarity:

📌 Creating Kubernetes Objects with Namespace & LimitRange – DEV-3

This lecture demonstrates **deploying objects into a namespace with a LimitRange**, verifying that the defaults are applied, testing the application, and cleaning up.

1️⃣ Apply All Manifests

Since the **namespace is already defined in the manifests**, you don't need `-n dev3` anymore:

```
kubectl apply -f cube-manifests/
```

- This creates:
  - **Namespace** (`dev3`)
  - **LimitRange** (CPU/memory defaults)
  - **StorageClass** (cluster-wide)
  - **PersistentVolumeClaim**
  - **ConfigMap**
  - **Deployments** (MySQL, User Management)
  - **Services**
  - **Secrets**

✅ Everything is created in the correct namespace automatically.

2️⃣ Verify Pods and Services

Check all pods in `dev3`:

```
kubectl get pods -n dev3
```

Check all services:

```
kubectl get svc -n dev3
```

- **MySQL** and **User Management Microservice** should be running.

- Liveness and readiness probes are automatically managed.

---

3 Inspect YAML of Pods

To verify that **LimitRange defaults** are applied automatically:

```
kubectl get pod <pod-name> -n dev3 -o yaml
```

- Look under `spec.containers[].resources`:

```
resources:
  limits:
    cpu: 500m
    memory: 512Mi
  requests:
    cpu: 300m
    memory: 256Mi
```

- Both the **init container** and **main container** inherit these defaults.
- This confirms that the **LimitRange is applied per container automatically**.

---

4 Inspect LimitRange

List and describe the LimitRange in the namespace:

```
kubectl get limits -n dev3
kubectl describe limits <limit-range-name> -n dev3
```

- Shows **default CPU/memory** and **default requests** applied to all containers.

  ⚠️ Remember: `Limits` is case-sensitive; use `limits` or `Limits` depending on your kubectl version.

---

5 Test the Application

Access the service:

```
kubectl get svc -n dev3
```

- Note the `NodePort` (e.g., `30002`)
- Test via browser or curl:

```
http://<node-ip>:30002/
```

- Response should be as expected, confirming the microservices are up.

6️⃣ Cleanup

Two options to clean everything:

1. **Delete via namespace:**

```
kubectl delete ns dev3
```

- Deletes all objects inside the namespace, including LimitRange, PVCs, ConfigMaps, Deployments, Services, etc.

2. **Delete via manifest:**

```
kubectl delete -f cube-manifests/
```

- Deletes objects according to manifests, including namespace and LimitRange.

  StorageClass is cluster-scoped; it won't be deleted with the namespace and must be removed manually if needed.

✅ Key Takeaways

- LimitRange automatically enforces CPU/memory defaults for all containers in a namespace.
- Namespace must exist before other resources; manifests should include `namespace` in metadata.
- Using declarative YAML allows **one command deployment** (`kubectl apply -f`) for all resources.
- Cleaning up is simplified by deleting the namespace.

Here's a clear, structured summary of your lecture on **Kubernetes Resource Quotas** and how it works with namespaces:

📌 Kubernetes Resource Quotas – DEV-3 Namespace

Resource Quotas let you **limit the total amount of resources** a namespace can consume. This is different from LimitRange, which sets **per-container defaults**.

---

1 What Resource Quotas Do

- Apply **per namespace**.
- Limit resources like:
  - **CPU** and **memory** (requests & limits)
  - Number of **pods**
  - Number of **services**, **config maps**, **persistent volume claims**, **secrets**
- Ensures that no deployment in the namespace can exceed the defined **hard limits**.

---

2 Example Use Case

- Namespace: `dev3`
- Quota:
  - Maximum **5 pods**
  - Maximum **5 config maps**
  - Maximum **2 CPUs**, **2 Gi memory**

Behavior:

- If you try to scale beyond the defined quota (e.g., 10 pods), Kubernetes **won't schedule the excess pods**.
- Useful in QA, staging, or multi-tenant clusters to prevent resource hogging.

---

3 ResourceQuota Manifest Example

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: ns-resource-quota
  namespace: dev3
spec:
  hard:
    pods: "5"
    configmaps: "5"
    persistentvolumeclaims: "5"
    requests.cpu: "2"
```

```
requests.memory: 2Gi
limits.cpu: "2"
limits.memory: 2Gi
```

- `hard`: Maximum allowed usage for each resource in the namespace.

---

4 Applying Resource Quotas

```
kubectl apply -f 05-05-03/kube-manifests/
```

- Creates:
  - **Namespace `dev3`**
  - **LimitRange** (default container CPU/memory)
  - **ResourceQuota**
  - All other resources like Deployments, Services, PVCs, ConfigMaps.

---

5 Verifying Resource Quotas

1. List quotas in the namespace:

```
kubectl get quota -n dev3
```

2. Describe a specific quota:

```
kubectl describe quota ns-resource-quota -n dev3
```

Output Example:

Resource	Used	Hard
ConfigMaps	1	5
Pods	2	5
PersistentVolumeClaims	1	5
CPU (limits)	1	2
Memory (limits)	1Gi	2Gi

- Shows **current usage** vs **hard limit**.

---

6 Key Points

- ResourceQuota is **namespace-level**; LimitRange is **per-container**.
- Helps **enforce multi-tenant policies** and **prevent resource overuse**.
- Combined with LimitRange, you can **control both per-container defaults and total namespace usage**.

- Scaling deployments beyond the quota will **fail scheduling**, even if the pods request resources.

---

7 Cleanup

```
kubectl delete ns dev3
```

- Deletes all resources in the namespace, including the **ResourceQuota**.
- Cluster-scoped resources like **StorageClass** are not deleted automatically.

💡 **Next Steps Mentioned in Lecture:**

- Understanding why using **EBS with MySQL in Kubernetes** is not ideal.
- Exploring **RDS integration**.
- Accessing external services via **ExternalName Service**.